Contents

1 Project Proposal **3**
- 1.1 Project Title . . . 3
- 1.2 Project Team . . . 3
- 1.3 Problem Statement . . . 3
- 1.4 Target Users . . . 3
- 1.5 Project Aim . . . 4
- 1.6 Project Objectives . . . 4
- 1.7 Expected Outcomes . . . 5
- 1.8 Project Scope . . . 6
- 1.9 Engineering Complexity . . . 6

2 Evidence of Relevance **7**
- 2.1 Real-World Business Request . . . 7
- 2.2 Evidence from Academic Research and Industry Practice . . . 7
- 2.3 Relevance to the Proposed Diploma Project . . . 8
- 2.4 Conclusion . . . 9

3 Literature Review **9**
- 3.1 Fragmented rental records . . . 9
- 3.2 Evidence from Kazakhstan and the proposed workflow . . . 10
- 3.3 Estimating rental prices . . . 11
- 3.4 Data consistency and access control . . . 12
- 3.5 Review findings and the project problem . . . 12
  - Working problem statement . . . 12
- 3.6 Analytical comparison of scientific publications . . . 13
- 3.7 Technical documentation and practical evidence . . . 14

4 Existing System Comparison **14**
- 4.1 Purpose and Method . . . 14
- 4.2 Existing Solutions . . . 15
- 4.3 Proposed Operational Workflow . . . 15
- 4.4 Comparison Against Common Criteria . . . 15
- 4.5 Findings and Project Justification . . . 17

5 Technology Comparison **17**
- 5.1 Evaluation Criteria . . . 17
- 5.2 Frontend Technologies . . . 18
- 5.3 Backend Technologies . . . 19
- 5.4 Database Technologies . . . 20
- 5.5 Selected Stack and Implementation Implications . . . 21

6 Requirements Specification **22**
- 6.1 Purpose and Scope . . . 22
- 6.2 Users and Permissions . . . 22

6.3 Requirement Priorities . . . 23
6.4 Functional Requirements . . . 23
6.5 Non-Functional Requirements . . . 26
6.6 Basis and Verification . . . 27
6.7 Alignment with the Project Proposal . . . 28

7 Architecture and Diagrams 29
7.1 System Context Diagram . . . 29
7.2 Use Case Diagram . . . 29
7.3 Component Diagram . . . 30

8 Verification and Evaluation Plan 30
8.1 Purpose . . . 30
8.2 Functional verification . . . 30
8.3 Quality verification . . . 32
8.4 Baseline and evaluation tasks . . . 33
8.5 Outcome metrics . . . 34
8.6 Measurement conditions . . . 34
8.7 Test evidence and completion . . . 35

9 Project Management and Risks 36
9.1 Risk Management . . . 36
Risk Register . . . 36
Risk Mitigation Strategies . . . 36
Risk Monitoring and Control . . . 37
9.2 Project Milestones . . . 37
9.3 Project Dependencies . . . 38

References 39

1 Project Proposal

1.1 Project Title

Development of a Unified Information System for Automating Tenant, Lease Agreement, and Commercial Space Management.

1.2 Project Team

Idrissova Ayaulym – backend development and database design.

Seitkhankyzy Aruzhan – business logic development, analytical functions, and testing.

Muslimova Aluana – frontend development, UI/UX design, and ML module development.

Academic Supervisor: Nadirov Nurtas Konysuly.

1.3 Problem Statement

Currently, commercial property owners and managers have many different software applications available to help organize all aspects of their business. For example, Excel is used for managing spreadsheets and tracking premises, CRM systems for interacting with tenants, and 1C for accounting.

However, information about tenants, lease agreements, commercial spaces, and payments is often stored in different places. This situation creates difficulties in finding required information, monitoring vacant and occupied properties, tracking lease expiration dates, and identifying overdue payments.

In addition, when determining rental prices for commercial premises, managers often have to manually compare property characteristics and market offers, which can be time-consuming and make it difficult to estimate an appropriate rental price.

Therefore, there is a need to develop a unified information system that combines the main rental management processes in one platform, automates routine tasks, and provides additional tools for data analysis and rental price estimation using machine learning.

1.4 Target Users

The proposed system is intended for commercial property owners, property management companies, leasing managers, and administrative staff.

The system will be used to manage various types of commercial premises, including offices, retail spaces, warehouses, and other properties available for rent.

1.5 Project Aim

The main aim of this diploma project is to develop a web-based information system to automate the management of tenants, lease agreements, commercial spaces, and rental payments.

The system will enable centralized storage and processing of information, monitoring of available and occupied spaces, tracking of lease agreement expiration dates and outstanding payments, automatic generation of notifications, and provision of analytical indicators for commercial property rental management.

Additionally, the project aims to develop and integrate a custom machine learning (ML) module to predict the estimated monthly rental price of commercial spaces based on their characteristics, such as area, location, property type, floor, and renovation condition.

1.6 Project Objectives

1. Analyze user requirements and the main business processes involved in commercial property rental management.
2. Design the information system architecture and a relational database to store information about commercial properties, premises, tenants, lease agreements, and payments.
3. Develop the backend of the application using Java and Spring Boot, and implement REST APIs for communication between system components.
4. Develop a user-friendly web application interface using React, including forms for adding and editing data, as well as search and filtering functionality.
5. Implement modules for managing commercial properties, premises, tenants, and lease agreements, including displaying available and occupied spaces, automatically checking space availability, and preventing overlapping rental periods.
6. Develop functionality for tracking rental charges, recording payments, automatically calculating outstanding balances, and identifying overdue payments.
7. Implement automated monitoring of lease agreement expiration dates and generate notifications about expiring contracts and outstanding payments.
8. Develop an analytical Dashboard to monitor commercial space occupancy, analyze space utilization efficiency, property profitability, lease agreement status, and outstanding payments.
9. Develop and integrate a custom machine learning (ML) module to predict the estimated monthly rental price of commercial premises based on their

characteristics, including data preparation, model training, comparison, and evaluation.

10. Implement user authentication and role-based access control to ensure data security and confidentiality.
11. Integrate the frontend, backend, database, and ML module, and conduct functional, unit, and integration testing of the information system.

1.7 Expected Outcomes

As a result of this diploma project, a functional web-based information system is expected to be developed to automate the management of tenants, lease agreements, commercial spaces, and rental payments.

The developed system will include the following features:

1. **Centralized Database:** A database for storing and managing information about commercial properties, premises, tenants, lease agreements, and payments.
2. **Commercial Property and Premises Management Module:** A module that allows users to add, edit, search, and filter commercial properties and premises, as well as view available and occupied commercial spaces.
3. **Tenant Management Module:** A module for storing tenant contact information, viewing tenant details, and accessing related lease agreements.
4. **Lease Agreement Management Module:** A module for creating, editing, and tracking lease agreements, automatically checking space availability, and preventing overlapping rental periods.
5. **Rental Payment Management Module:** A module for recording rental charges and payments, automatically calculating outstanding balances, and identifying overdue payments.
6. **Automated Notification System:** A system that notifies users about upcoming lease agreement expiration dates and outstanding payments.
7. **Analytical Dashboard:** A dashboard displaying commercial space occupancy rates, space utilization efficiency, property profitability, lease agreement status, and outstanding payments.
8. **Custom ML Module for Rental Price Prediction:** A machine learning module trained on a prepared dataset to estimate the monthly rental price of commercial premises based on characteristics such as area, location, property type, floor, and renovation condition. The model's performance will be evaluated using MAE and RMSE metrics.

9. **User Authentication and Role-Based Access Control:** A system that provides secure user login and access to system features according to assigned user roles.
10. **Integrated Web Application:** A fully integrated application connecting the frontend, backend, database, and ML module through REST APIs.
11. **Testing Results and Technical Documentation:** Documentation and testing results covering the main system functions, calculation accuracy, component interactions, and ML model performance.

The final outcome of the project will be a functional prototype of an information system designed to simplify commercial property rental management, reduce manual data processing, and support decision-making through analytics and machine learning.

1.8 Project Scope

The project will include:

The development of a web-based information system for managing tenants, commercial properties, lease agreements, and rental payments. The system will include centralized data management, automated notifications, space availability monitoring, an analytical dashboard, and a machine learning module for estimating rental prices.

Rental payment information will be recorded and managed within the system, while official accounting operations will remain in 1C.

The project will not include:

Mobile application development, online payment processing, full-scale accounting functionality, direct integration with 1C, or the development of a large language model from scratch.

The ML module will be developed using existing machine learning algorithms, which the team will independently train, compare, and evaluate on a prepared dataset.

1.9 Engineering Complexity

The technical complexity of the diploma project lies in developing a web-based information system consisting of several interconnected components.

The backend will handle user requests, manage data, implement authentication and role-based access control, and execute business logic. The relational database will store information about commercial properties, premises, tenants, lease agreements, and rental payments.

The user interface will provide convenient interaction with the system through forms, tables, search and filtering functions, and visualization of analytical data.

A separate technical challenge will involve implementing automated lease expiration monitoring, outstanding balance calculations, and space availability checks to prevent overlapping lease agreements for the same premises. The system will also provide analytical functions for monitoring occupancy rates, rental charges, received payments, and outstanding balances.

An additional challenge will be the development of a custom machine learning module for estimating monthly rental prices based on property characteristics. This process will include data collection and preprocessing, training and comparing Linear Regression and Random Forest models, evaluating model performance on unseen test data using MAE and RMSE, and integrating the selected model into the web application.

Thus, the diploma project will combine full-stack web development, relational database design, implementation of rental management business logic, software testing, and the practical application of machine learning methods. The developed system is expected to reduce manual data processing, improve rental management efficiency, and support decision-making through automated analytics and rental price estimation.

2 Evidence of Relevance

2.1 Real-World Business Request

The relevance of the proposed diploma project is supported by a real business request published on the Astana Hub platform under the title *Development of CRM/WMS* [14].

The request concerns the development of a unified information system for managing commercial warehouse and industrial properties in Kazakhstan. According to the client's description and technical specification, the main problem is the fragmentation of rental management information across separate files and accounting systems, including Excel and 1C.

The client identifies the need to centralize information about commercial premises, tenants, lease agreements, rental charges, and outstanding payments. The proposed solution is also expected to provide occupancy monitoring, lease expiration tracking, role-based access control, and automated management reporting [14].

This business request provides practical evidence of an existing need for a centralized information system to support rental management processes.

2.2 Evidence from Academic Research and Industry Practice

The relevance of the identified problem is further supported by academic research and practical experience in property management automation.

Bassett and Pisano [1] examine the fragmentation of technological solutions in property management. Their study highlights how the use of multiple software applications can result in repeated data collection and additional administrative work. The authors also explain that integrated systems can reduce duplicate data entry and improve the efficiency of rental management processes.

Pomè, Tagliaro, and Lavy [2] investigate the gap between existing PropTech solutions and the actual needs of property management professionals. Their study includes a survey of 45 industry professionals and interviews with PropTech companies. The findings identify difficulties in integrating digital technologies into daily operations and highlight the importance of centralized data management and automated monitoring of contractual deadlines.

A practical example from Kazakhstan is presented in the Dars-2 implementation case [13]. The case describes the automation of management accounting processes in a company involved in office property rental in Almaty. It illustrates practical challenges related to manual accounting operations and the management of rental information.

Together, these sources demonstrate the importance of reducing fragmented information management, improving access to operational data, and automating repetitive rental management tasks.

2.3 Relevance to the Proposed Diploma Project

The proposed Commercial Property Rental Management System is based on the main problem identified in the Astana Hub business request and adapted to different types of commercial properties, including offices, retail premises, and warehouses.

The system aims to centralize information about properties, tenants, lease agreements, rental charges, and payments within a single web application. Its main functionality will include space availability monitoring, lease agreement management, outstanding balance calculations, automatic notifications, and analytical dashboards.

In addition, the project will include a machine learning module for estimating monthly rental prices based on property characteristics. This module represents an additional functionality proposed by the project team.

Unlike the original warehouse-focused request, the proposed system will focus on the main rental management processes applicable to different types of commercial real estate. Full accounting functionality and direct integration with 1C are outside the project scope.

The project team has identified potential clients and is currently in the process of establishing contact to discuss their requirements and the possibility of conducting interviews.

2.4 Conclusion

The relevance of the proposed diploma project is supported by a real business request published on Astana Hub [14], academic research on fragmented property management technologies [1], research on the gap between digital solutions and industry needs [2], and a documented automation case in Kazakhstan [13].

These sources provide both practical and academic justification for developing a unified information system that centralizes rental management data and automates essential business processes.

The proposed project aims to address the identified problems and improve the efficiency of commercial property rental management.

3 Literature Review

3.1 Fragmented rental records

This review examines the need for a shared operational record of commercial premises, tenants, lease agreements, charges, payments and outstanding balances. The proposed platform will support daily rental management, while bookkeeping will remain in a separate 1C system. The review draws on eight scientific publications, four sources of official technical documentation and one published implementation case from Kazakhstan. It connects these sources to the project problem, system boundaries and evaluation plan. It is a thematic review, not an exhaustive systematic review.

Bassett and Pisano (2022) examine the connected processes of property management, including leasing, rent collection and renewals [1]. They explain how integrated software can reuse information collected earlier in the rental process and avoid entering it again. This supports keeping tenant, property, agreement and payment records connected. However, the paper does not measure the reduction in errors through a controlled experiment and focuses largely on residential property. Its contribution is the explanation of why disconnected records create unnecessary work.

Pomè, Tagliaro and Lavy (2026) examine how well digital tools meet the needs of property managers in Italy [2]. They review 29 solutions, study four in detail and analyse 45 responses from professionals. They identify differences between the functions offered and users' working practices. Unlike Bassett and Pisano, this study compares the perspectives of suppliers and staff. Its limitations include a single national context, a small sample and purposive case selection. For our project, it supports validating work scenarios before selecting interface functions. A large number of features does not, by itself, demonstrate that an application is useful.

Wang (2023) reviews 22 publications on smart real estate [3]. The review provides context for digital information management and data analysis, but it does

not demonstrate the effectiveness of a specific rental application. Taken together, these studies support examining how information moves through rental operations. They do not prove that an additional application will automatically improve an organisation's work. The local case below makes the operational problem more concrete.

3.2 Evidence from Kazakhstan and the proposed workflow

The published Dars-2 case concerns an office landlord in Almaty operating two business centres with a total area of 12,000 m^2 and serving more than 100 clients [13]. According to the implementation report, the company used 1C:Accounting for Kazakhstan 2.0, while available space and the allocation of utility costs were maintained in Excel. Availability information was collected from managers daily. Agreement terms were not recorded in the accounting application, so changes required checking physical contracts. Management lacked a current view of available premises and tenant debt.

The company subsequently adopted 1C:Accounting for Kazakhstan 3.0 with the BIT.Arenda module [13]. This is a vendor-reported case from 2021, not an independent scientific study or evidence that the same conditions affect all landlords in Kazakhstan. Nevertheless, it documents a local example of the specific information gap addressed by our project. It also shows that established solutions exist. Our justification therefore concerns a defined operational workflow, not an absence of rental software in Kazakhstan.

Our proposed system is a single platform for operational management of commercial rentals. It links premises, tenants, lease agreements, charges, registered payments and outstanding balances. For example, a manager should be able to check whether a commercial unit is available for a requested period, open its agreement and see the associated charges and recorded payments without combining separate operational spreadsheets. Availability must be derived from agreement dates and statuses, rather than from a single permanent "occupied" flag.

Accounting will continue separately in 1C. The accountant will verify actual incoming funds using bank records or 1C and then manually register the confirmed payment in our platform. Each payment should identify its date, amount, supporting reference and allocation to an agreement or charge. The platform will calculate the remaining balance from its own charges and registered payments. This balance can differ from the bank or 1C until the accountant records and reconciles the relevant transactions; it must not be presented as automatically synchronised financial information.

Direct integration with 1C is outside the project scope. Export to Excel or CSV is considered an optional function for later information exchange with external accounting systems. An export file alone does not provide automatic import, reconciliation or two-way synchronisation. Unlike the integrated solution in the Dars-2 case, our application will retain a manual boundary between operational

and accounting records. Its expected benefit is a consistent view within rental operations, while duplicate entry across the two systems remains a limitation.

3.3 Estimating rental prices

Mohd, Harussani and Masrom (2022) compare Random Forest, Decision Tree and Support Vector Machine models for estimating office rents in Kuala Lumpur [4]. They use data from 2015–2021 and 21 features. Table VI reports $R^2 = 0.906$ for Random Forest with cross-validation. This result makes Random Forest a suitable candidate for comparison, but it does not guarantee the same performance in our project. The authors work with a specific market and set of characteristics. Moreover, R^2 should not be interpreted as "90.6% accuracy" for individual predictions.

In a subsequent study, Mohd et al. (2023) focus on office location [5]. Using Kuala Lumpur data from 2018–2023, they examine proximity to the central business district, transport, surrounding amenities and traffic conditions. The authors highlight the importance of proximity to the business district and note that results depend on data quality. The study supports including location in the model, but it does not establish a universal price premium. These two studies partly share authors and geographic coverage, so they should not be treated as independent evidence that the method generalises across markets.

Deppner and Cajias (2024; first published online in 2022) show why real estate model validation requires particular care [6]. They use 9,256 apartment rental listings in Frankfurt and compare random and spatial data splits. With random splitting, nearby properties may appear in both training and validation sets, making performance appear better than when the model is applied to new data. The study concerns housing, so its findings do not establish the accuracy of office rent estimates. Its contribution to this project is methodological: validation should reflect where and when predictions will be used.

Our project proposes comparing Linear Regression and Random Forest with a simple baseline that always predicts the median of the training set. DummyRegressor supports this strategy [9]. The median prediction helps establish whether a model improves on a simple estimate. According to the scikit-learn documentation, missing-value imputation and other learned transformations should be fitted only on the training data; Pipeline helps enforce this rule [10]. MAE and RMSE will be used for evaluation [11]. Both errors should be reported in the units of the target variable, for example tenge per month when predicting total monthly rent.

Before training, the city, time period and property types must be defined. If the data come from listings, the output will estimate asking rent rather than the rent in a completed lease agreement. Repeated listings for the same property must not appear in different data subsets. Where dates and building identifiers are available, the model should also be evaluated on later listings or on separate buildings. These are proposed measures for the project, not experiments

that have already been conducted. The final model can only be selected after comparison using our own data.

3.4 Data consistency and access control

Combining forms on one website is not sufficient for a system used by several employees. Berenson et al. (1995) analyse anomalies in concurrent transactions [8]. A possible project scenario is that two managers simultaneously see a property as available and create agreements for overlapping dates. This is our application of the paper's findings, not a case described by its authors. Validation must be enforced on the server and in the database. PostgreSQL documentation shows how range types and exclusion constraints can prevent overlaps [12]. The specific constraint must account for the property, date range and agreement statuses that reserve the space.

Sandhu et al. (1996) provide formal models for role-based access control [7]. These models support separating the responsibilities of the Property Owner, Rental Manager and Accountant. In our system, the owner manages staff access, the manager maintains tenants and agreements, and the accountant records verified payments. A tenant is a business record, not a system user. The specific permission matrix must be defined by the project team and validated with users. This foundational paper supports the access-control design; it is not evidence of current rental-market conditions.

Manual payment entry also requires controls inside the platform. Required references, positive amounts, checks for possible duplicate payments and a record of who entered or corrected a payment should make reconciliation easier. A partial payment must reduce only the balance to which it is allocated. These are proposed requirements derived from the selected workflow, not findings measured in the cited studies. They do not verify bank transactions automatically.

3.5 Review findings and the project problem

The sources support a specific operational problem. Bassett and Pisano explain the value of connected rental records [1]; Pomè et al. show why tools must match staff workflows [2]; and the Dars-2 case documents gaps in availability, agreement and debt information in a Kazakhstani company [13]. These sources justify investigating an operational platform alongside separate accounting. They do not establish that our implementation is superior to existing products or that it will eliminate all manual work.

Working problem statement

Commercial landlords who maintain premises, tenant details, agreement terms and financial records separately may lack a consistent operational view of rental activity. Managers must reconcile records to establish whether a commercial unit is available for a particular period, which agreement applies and what

amount remains outstanding. The project addresses this fragmentation by linking premises, tenants, agreements, charges and manually registered confirmed payments in one operational platform. Accounting remains in 1C, and payment information becomes current in the platform only after verification and entry by the accountant. The intended outcome is more reliable daily rental control, rather than replacement of accounting or automatic synchronisation with it.

The Dars-2 report provides a documented local example, but the workflow of the intended users still needs confirmation through three to five interviews or a substantive client discussion. This should establish where records are kept, how frequently information is updated, how payments are verified, and which tasks require manual reconciliation. This review does not include primary interview data or measured error rates and time savings.

The research question is whether a shared operational platform can improve the speed and correctness of availability checks, agreement management and outstanding-balance calculations while accounting remains separate. A second question concerns the usefulness of an indicative monthly rent estimate based on property characteristics. ML is an additional decision-support function; it does not resolve missing payment entries or inconsistent agreement records.

Evaluation should use defined tasks: checking availability for a period, creating an agreement, rejecting an overlapping agreement and registering a confirmed partial payment against a charge. Task completion time and error counts can be compared with the users' existing method under equivalent conditions. A reconciliation task should also test whether omitted or duplicate payment records can be identified. ML will be evaluated separately against the median baseline using MAE and RMSE. These are planned evaluations; no performance improvement is claimed at this stage.

3.6 Analytical comparison of scientific publications

Source	**Method and contribution**	**Limitation**	**Use in the project**
Bassett and Pisano, 2022 [1]	Analysis of rental processes and fragmented software.	No controlled measurement of impact; mainly residential property.	Link tenants, agreements and financial records.
Pomè et al., 2026 [2]	29 solutions, four cases and 45 professional responses.	Italy; purposive selection and a limited sample.	Validate required features and usability through interviews.
Wang, 2023 [3]	Bibliometric analysis of 22 publications.	Does not evaluate a completed application.	Context for digital management and data analysis.

Source	Method and contribution	Limitation	Use in the project
Mohd et al., 2022 [4]	Comparison of three office rent prediction models.	One market; generalisability is not established.	Include Random Forest in the model comparison.
Mohd et al., 2023 [5]	Assessment of location factors in office rents.	Local data; other price factors also matter.	Evaluate location features using our own data.
Deppner and Cajias, 2024 [6]	Random and spatial validation using housing listings.	Frankfurt housing market; validation depends on the task.	Separate related properties and assess generalisation.
Sandhu et al., 1996 [7]	Formal role-based access control models.	Do not define an organisation's specific permissions.	Permission matrix for three roles and access checks.
Berenson et al., 1995 [8]	Analysis of transaction isolation and anomalies.	Not a study of rental management systems.	Test concurrent agreement creation.

3.7 Technical documentation and practical evidence

Source	Purpose
DummyRegressor [9]	Median baseline for comparing predictive performance.
Common pitfalls [10]	Preventing data leakage and ensuring consistent preprocessing.
Metrics and scoring [11]	Definitions of MAE and RMSE for model evaluation.
PostgreSQL Range Types [12]	Storing periods and preventing overlaps in the database.
Dars-2 implementation case [13]	Kazakhstani example of separate availability, contract and payment records. Vendor report, not a scientific publication.

The four documentation sources support implementation and evaluation choices but are not scientific articles. The Dars-2 case is used to ground the operational problem in Kazakhstan and is not included in the count of eight scientific publications.

4 Existing System Comparison

4.1 Purpose and Method

This comparison examines three existing commercial property management products against nine shared criteria: premises and occupancy; tenants; lease agreements; charges and payments; notifications; analytics; user roles;

accounting boundary and data exchange; and rental price estimation. The criteria reflect the operational workflow and decision-support needs described in the updated literature review.

The products are 1C Rental and Property Management, Yardi Voyager Commercial and Rent Manager. They are relevant because they combine rental records with financial or accounting functions. The comparison uses official product descriptions, reviewed on 21 October 2026; the products have not been tested. NC means not confirmed in the reviewed materials, not that a capability is absent. The proposed platform column describes planned functions, not implemented or measured results. Different role names or an NC entry do not establish a product limitation; permissions and model capabilities require further verification.

4.2 Existing Solutions

1C Rental and Property Management supports commercial property registers, premise statuses, lease documents and rental settlements within a 1C-based solution. It is relevant as an example of combining rental operations and accounting. It must not be confused with every configuration of 1C or assumed to be identical to the Kazakhstan deployment discussed in the literature review.

Yardi Voyager Commercial combines commercial property and lease information with accounting and analytics. Its documented scope includes office, retail and industrial property. It is relevant as a broad platform connecting portfolio operations and financial management.

Rent Manager supports commercial lease changes, charges, payment receipt and reporting. Its accounting functions and API provide a useful comparison with a platform that retains a separate accounting system.

4.3 Proposed Operational Workflow

The proposed web platform links premises, tenants, agreements, charges, registered payments and outstanding balances. The Accountant verifies actual receipts using bank records or 1C and manually records confirmed payments in the platform, identifying their date, amount, supporting reference and allocation. Balances are calculated from the platform's own records and depend on timely entry. Direct integration with 1C is outside scope. Excel/CSV export is a possible additional function; it does not provide automatic import or synchronisation.

4.4 Comparison Against Common Criteria

Criterion	1C Rental and Property Management	Yardi Voyager Commercial	Rent Manager	Proposed Platform
Premises and occupancy	Property register, period statuses, occupancy and area checks on new leases.	Central property database; exact availability rules: NC.	Commercial property records; exact overlap prevention: NC.	Offices, retail and warehouses; availability from agreement dates and statuses; overlap checks.
Tenants	Tenant-linked lease and settlement records.	Tenant and lease information within commercial operations.	Tenant charges and commercial lease records.	Tenant profiles linked to premises, agreements, charges and recorded payments.
Lease agreements	Lease creation, amendments and termination documents.	Centralised commercial lease terms and administration.	Add, edit, renew and export commercial leases.	Create, update and track agreements; validate dates and premise availability.
Charges and payments	Rental settlement management alongside accounting.	Built-in accounting; project-specific balance rules: NC.	Receivables include charges, payment receipt and deposits.	Accountant manually enters verified payments; balances and arrears use registered records.
Notifications	Automatic lease expiry reminders; exact arrears-alert rules: NC.	Critical-data notifications; exact reminder timing: NC.	Project-specific reminder rules: NC.	Lease expiry and unpaid overdue charge notifications.
Analytics	Property dashboards and status charts.	Performance analytics and role-based dashboards.	Commercial Rent Roll and custom reports.	Occupancy, vacancies, agreement status, charges, receipts and unpaid balances.
User roles	Detailed access permissions: NC in reviewed page.	Role-based dashboards; detailed access rules: NC.	Detailed access permissions: NC in reviewed page.	Property Owner, Rental Manager and Accountant; tenants are managed records.
Accounting and exchange	Built on a 1C accounting configuration; local compatibility needs checking.	Accounting included; compatibility with a specific 1C configuration: NC.	Accounting and an API; a specific 1C connector: NC.	Accounting stays in separate 1C; no direct integration. Optional future Excel/CSV export.
Rental price estimation	Trained premise-level monthly-rent estimation: NC.	Trained premise-level monthly-rent estimation: NC.	Financial forecasts are described; equivalent trained rental-price model: NC.	Planned Linear Regression/Random Forest comparison; MAE/RMSE and median baseline.

Table 3: Comparison of existing systems and the proposed platform

4.5 Findings and Project Justification

All three existing systems address substantial parts of commercial rental management. Centralised records, agreement administration and payment tracking are therefore not unique project features. Vendor documentation also does not establish that these products are unsuitable for the intended users. Their actual fit would require demonstrations and testing against the same scenarios.

The project has a narrower operational scope: staff work in one rental platform while accounting remains separate. This matches the workflow described in the updated literature review, in which the accountant verifies receipts externally and then registers them manually. The intended contribution is a linked operational view of premises, tenants, agreements, charges and confirmed payment records, supported by permissions for three staff roles.

The manual boundary is also a limitation. A bank receipt or 1C transaction does not update the platform automatically. Missing, delayed or duplicate entries can make its outstanding balance differ from accounting records. Payment references, allocation checks and traceable corrections support reconciliation, but they do not replace the accountant's verification. Optional Excel/CSV export must not be described as integration.

The proposed rental price model is a separate decision-support feature. Its value must be tested on the project dataset against a median-rent baseline using MAE and RMSE. Unconfirmed equivalent models in the compared products do not prove that the project is unique or more accurate.

The expected operational benefit is less searching across disconnected records and more consistent availability and balance checks within the platform. This remains a hypothesis. Evaluation should compare the application with the users' current Excel workflow using equivalent data and tasks, recording completion time, correctness and assistance. A reconciliation task should also check omitted or duplicate payment entries. The comparison does not claim that the project removes double entry across accounting and rental operations.

5 Technology Comparison

5.1 Evaluation Criteria

The technologies are compared against the operational requirements of the commercial property rental management system. The frontend must support linked forms, searchable tables and consistent updates across screens. The backend must enforce access permissions, validate lease periods and process charges and confirmed payments. The database must preserve relationships between records and prevent conflicting reservations of the same premises.

The comparison considers functional suitability, data integrity, maintainability and implementation effort. Official documentation is used to examine the capa-

bilities and limitations of each alternative. Accounting remains in a separate 1C system, while verified payments are entered manually into the rental platform. Direct integration with 1C is therefore not a selection requirement.

5.2 Frontend Technologies

React, Vue and Angular are considered for the user interface. All three can support reusable components and interactive forms. For this project, the main distinction is how form state and shared information are organised, and how many conventions the development team must define [15, 16, 17].

Table 4: Comparison of frontend technologies

Criterion	**React**	**Vue**	**Angular**
Form state	Component state represents values, errors and submission status.	Reactive state and template bindings support form interaction.	Reactive forms provide explicit controls, groups and validation.
Shared data	State can be shared through common parent components, context and reducers.	Reactive stores support shared state; Pinia is recommended for larger applications.	Shared services can organise data used by several components.
Implementation effort	Routing, API handling and validation conventions must be selected.	Routing and shared-state tools are added according to application needs.	More application conventions are supplied by the framework.
Project trade-off	Flexible component design, with greater responsibility for application conventions.	A viable alternative with template-based components and reactive data binding.	Structured form support, with more framework conventions to adopt.

React is selected for the planned interface. Its state model supports workflows in which users select premises, choose a tenant and submit a lease agreement. Reusable components can provide consistent tenant selectors, date fields, payment forms and validation messages. The same capabilities are available in Vue and Angular; the selection does not depend on an assumed performance advantage.

The main trade-off is the need to establish common conventions for requests, form validation and shared data. React documentation identifies redundant state as a source of errors [15]. In this system, the frontend should therefore retrieve updated financial values from the backend after a successful payment registration rather than maintain an independent calculation of the outstanding balance.

Vue would also meet the interface requirements. Angular offers a more explicit form structure, which could be useful if complex forms became a dominant part of the application [17]. The current requirements do not establish a decisive reason to replace the React approach already specified in the project proposal.

5.3 Backend Technologies

Spring Boot with Java, Django with Python, and Express on Node.js are compared as backend alternatives. The evaluation focuses on transaction management, access control and the organisation of database operations. These mechanisms are relevant to lease creation and payment registration, where a request may affect several related records.

Table 5: Comparison of backend technologies

Criterion	**Spring Boot**	**Django**	**Express**
Transactions	Spring supports declarative transaction boundaries around service operations.	Django provides atomic blocks through its database layer.	Transaction handling depends on the selected database driver or ORM.
Access control	Spring Security supports request-level and method-level authorisation.	The authentication system provides groups and permissions.	Permission checks are implemented through middleware and supporting libraries.
Persistence	Database access is configured through the selected Spring persistence approach.	An integrated ORM provides a conventional database access layer.	A database driver or ORM is selected separately.
Project trade-off	Fits the planned Java service layer, but requires security and persistence configuration.	Could share Python with the estimation module, while still requiring a custom rental API.	Shares JavaScript with the frontend, but requires more explicit library choices.

Spring transaction management allows related database changes to be grouped within a defined operation [18]. Spring Security also supports authorisation at service-method level, although this functionality must be explicitly enabled and configured [19]. These mechanisms suit a backend in which lease creation and payment registration are implemented as business services.

Django is a credible alternative because it provides transaction management and a permission system [20, 21]. Using Python for both the backend and the estimation module could reduce the number of implementation languages. Express provides greater freedom in selecting database and middleware components, but requires the team to establish a consistent application structure [22].

Spring Boot is retained because its transaction and authorisation mechanisms

fit the proposed service responsibilities and the planned Java backend. This is a project-specific decision rather than evidence that Spring Boot is inherently faster or more secure than the alternatives.

A transaction ensures that the database changes included in an operation are committed or rolled back together. It does not establish that an entered payment is genuine or that the business calculation is correct. The application must separately validate amounts, check permissions and apply the required database constraints. Concurrent requests also require a deliberate locking or constraint strategy.

5.4 Database Technologies

PostgreSQL, MySQL with InnoDB, and MongoDB are evaluated for storing rental records. The main requirement is to maintain consistent relationships between premises, tenants, agreements, charges and payments. A further requirement is to prevent overlapping reservations when several employees work with the same premises.

Table 6: Comparison of database technologies

Criterion	**PostgreSQL**	**MySQL with InnoDB**	**MongoDB**
Data model	Relational tables suit connected rental records.	Relational tables suit the same relationships.	Documents require an embedding or referencing strategy.
Atomic updates	Transactions group related changes.	Transactions group related changes.	Single-document writes are atomic; multi-document transactions are also available.
Period conflicts	Range types and exclusion constraints can enforce non-overlapping periods.	Conflict prevention requires a transactional check with appropriate locking.	Conflict prevention requires coordinated writes or an atomic reservation design.
Reporting	SQL joins and aggregation suit linked rental and payment reports.	SQL joins and aggregation support the same reporting needs.	Aggregation must follow the chosen document structure.
Project trade-off	Direct support for the period rule, with database-specific constraint design.	A suitable relational alternative, with an explicit locking protocol to maintain.	Additional decisions about references, duplication and concurrency control.

PostgreSQL is selected because its range types and exclusion constraints provide a direct way to prohibit overlapping periods for the same premises [23]. This

addresses a specific concurrency problem: two managers may both see a unit as available before either agreement has been saved. A database constraint can reject the conflicting write even when both requests pass an earlier availability check.

The constraint must reflect the rental rules. The implementation must define which agreement statuses reserve premises and whether the final date is inclusive. A half-open interval includes the start date and excludes the end date, allowing consecutive agreements to meet at a boundary. If the interface uses an inclusive final day, that value must be converted consistently before the interval is stored.

MySQL remains a viable alternative. InnoDB supports locking reads that can be used within a transaction [24]. One possible implementation is to lock the premises row, check for conflicting agreements using an appropriate current read, and insert the agreement before releasing the lock. All relevant write paths must follow the same protocol. A simple availability query followed by an uncoordinated insert would not provide equivalent protection.

MongoDB supports multi-document transactions and should not be excluded on the assumption that it lacks transactional behaviour [25]. However, the proposed system has stable relationships and reports that combine several record types. PostgreSQL fits this structure and expresses the period constraint more directly. The trade-off is greater dependence on database-specific functionality.

5.5 Selected Stack and Implementation Implications

The selected stack consists of React, Java with Spring Boot, and PostgreSQL. React provides the interactive interface, Spring Boot supports the application services, and PostgreSQL stores the linked rental records and enforces database constraints. The decision is based on functional suitability and implementation requirements rather than measured performance rankings.

Financial values should use exact decimal storage and a documented rounding rule. At charge level, the outstanding balance is the charge amount minus confirmed payments allocated to that charge. For example, a charge of KZT 300,000 and an allocated payment of KZT 120,000 produce an outstanding balance of KZT 180,000. The agreement screen and financial report must apply the same calculation.

Because 1C remains separate, the platform reflects payments registered by the accountant after verification. Its balance may differ from accounting records until reconciliation is completed. Optional Excel or CSV export can support this process, but it does not provide automatic synchronisation.

The implementation should be verified through concurrent lease requests, partial payments, repeated payment submissions, interrupted operations and unauthorised API requests. These checks establish whether the selected technologies have been configured and used correctly.

The choice between Linear Regression and Random Forest remains a separate empirical task. Both models should be evaluated on held-out data using MAE and RMSE against a median baseline. No final estimator is selected before this evaluation, and unavailability of the estimation module must not interrupt ordinary rental management.

6 Requirements Specification

6.1 Purpose and Scope

The web application centralises information about commercial properties, premises, tenants, lease agreements, rental charges and payments. It supports offices, retail spaces and warehouses, monitors occupancy and arrears, generates notifications, provides rental revenue analytics and estimates monthly rental prices using machine learning.

The Accountant verifies actual receipts in bank records or 1C and manually registers confirmed payments in the platform. Official accounting remains in a separate 1C system. Direct integration with 1C and automatic bank synchronisation are outside scope. Excel/CSV export is an additional Should function. Online payment processing, mobile application development, full accounting functionality and operating expense management are outside the project scope.

The implementation uses a React web interface, a Java/Spring Boot backend, a PostgreSQL database and a separate ML service connected through REST APIs. Central records are persisted in the database. The prototype will include unit, functional and integration tests and technical documentation.

6.2 Users and Permissions

Role	**Main permissions**
Property Owner	Manage users and roles, properties and premises. View agreements, occupancy, charges, registered receipts, arrears and rental revenue reports. Use rental price estimates.
Rental Manager	Manage premises, tenants and lease agreements. Search available premises. View occupancy, arrears and lease notifications. Use rental price estimates.
Accountant	Maintain rental charges and verified payment records. View tenants, agreements and outstanding balances. Manually reconcile platform records with bank or 1C evidence; view rental financial reports and export data if available.

Table 7: System Users and Permissions

Tenants are records managed by staff and do not have a separate application role. Permissions are checked by the backend, including direct API requests.

6.3 Requirement Priorities

Must requirements form the mandatory prototype scope. Should identifies an additional function that may be implemented after the mandatory scope. FR11 is Should; all other FR and NFR requirements below are Must. Each requirement has a corresponding check in the Verification and Evaluation Plan.

6.4 Functional Requirements

FR01 Authentication and Access (Must)

The system shall support login, logout and access according to the three roles. The Property Owner shall manage user accounts and role assignments.

Acceptance criterion. Valid credentials permit login; invalid credentials are rejected. Logout ends access. A blocked account cannot access protected operations. The backend rejects actions outside the assigned role, including account administration by the Rental Manager or Accountant.

FR02 Properties and Premises (Must)

The system shall allow the Property Owner to manage properties and the Property Owner and Rental Manager to manage their premises.

Acceptance criterion. A property stores its name, address, location and property type. Each premise has a property reference, code, type, positive area and availability information. Duplicate codes within one property and invalid areas are rejected. Unavailable periods have start and end dates and are excluded from availability searches. A conflicting unavailable period and confirmed agreement cannot both be saved. Saved changes appear in the premise list.

FR03 Tenant Records (Must)

The Rental Manager shall create, view and update tenant profiles.

Acceptance criterion. A tenant profile stores a name and contact information. Missing names or all contact details are rejected. The profile displays its related lease agreements. Existing agreement and payment references are preserved when tenant details change.

FR04 Search and Filtering (Must)

The system shall provide search and filtering for properties, premises, tenants and agreements.

Acceptance criterion. Users can find properties by name or address and records by tenant name, premise code and agreement number. Properties can be filtered by location and property type. Premises can be filtered by property, type and availability for a selected period. Returned records match the selected conditions.

FR05 Lease Agreement Management (Must)

The Rental Manager shall create, update and track lease agreements.

Acceptance criterion. An agreement stores its number, tenant, premise, start and end dates, monthly rent in KZT and status. Duplicate numbers, missing references, non-positive rent and an end date before the start date are rejected. A draft does not reserve space; a confirmed agreement reserves it for its date range. Expiry follows the recorded end date. Saved changes are visible in the agreement record.

FR06 Availability and Overlap Prevention (Must)

The system shall prevent confirmed agreements from occupying the same premise during overlapping periods.

Acceptance criterion. Lease start and end dates are inclusive. For a lease covering 1 to 31 March, another lease starting on 31 March is rejected; one starting on 1 April is allowed if otherwise available. The check applies to creation and changes. Two concurrent conflicting requests cannot both succeed.

FR07 Verified Payment Registration (Must)

The Accountant shall maintain rental charges and manually register externally verified payments. Payments shall be allocated to their agreement and charges; entries and corrections shall be traceable.

Acceptance criterion. A charge has an agreement, amount and due date. A payment has a stable ID, date, positive KZT amount, supporting bank or 1C reference and agreement. Allocations cannot exceed the payment or unpaid charge. Repeated submission does not duplicate a payment; a reused supporting reference raises a duplicate warning. Each entry or correction preserves author, time and changed values. A charge or external receipt alone does not register a payment.

FR08 Outstanding Balances and Arrears (Must)

The system shall calculate unpaid balances and overdue amounts from charges and allocated payments.

Acceptance criterion. A charge amount must be positive. For a charge of 100,000 KZT and a payment of 60,000 KZT, the unpaid balance is 40,000 KZT. If due on 10 March, that unpaid amount becomes overdue on 11 March. Partial payments reduce arrears. Overpayments are shown separately rather than as negative arrears. Only registered payments affect the balance. An unentered external receipt leaves the balance unchanged until the Accountant records it. The interface identifies the result as an operational balance based on registered records, not a synchronised 1C balance.

FR09 Lease and Payment Notifications (Must)

The system shall automatically monitor agreement expiry and unpaid due charges and generate internal notifications on a configured schedule.

Acceptance criterion. Without a user initiating a check, the scheduled process generates an expiry notification for an agreement entering the configured reminder period. Test evidence records the schedule, reminder period, control date and timezone. An unpaid charge produces an overdue notification after its due date. Each notification identifies the relevant agreement or charge. Repeated checks do not duplicate the same event, and visibility follows role permissions.

FR10 Occupancy and Rental Revenue Analytics (Must)

The system shall provide a dashboard showing occupied and vacant premises, area utilisation, agreement status, rental charges, registered rental receipts and outstanding balances. The Property Owner and Accountant shall be able to view rental revenue reports. The system shall not maintain operating expenses or calculate accounting profit or property profitability.

Acceptance criterion. For a selected date, occupied area is calculated from confirmed lease periods and premise areas. Rentable area excludes areas marked unavailable; vacant rentable area equals rentable area minus occupied area. For 100 square metres of rentable area and 40 occupied square metres, occupancy is 40% and vacant rentable area is 60 square metres. If rentable area is zero, the occupancy percentage is displayed as N/A. Rental charges and registered receipts are reported separately for a selected period, and outstanding balances are calculated according to FR08. Receipts are grouped by actual payment date, not entry timestamp; each payment is counted once and linked to its property through the agreement and premise. Reports must respect property and date filters. Unentered external bank or 1C receipts do not affect platform totals. Dashboard labels distinguish recorded rental receipts from charges and outstanding balances; no figures are labelled as net profit or profitability.

FR11 Optional Data Export (Should)

The Accountant should be able to export recorded payments for a selected period to Excel or CSV for later information exchange.

Acceptance criterion. The export contains payment ID, date, agreement reference, amount, currency and supporting reference. Record count and totals match the selected records; payment IDs stay stable across repeated exports. Export does not alter records, provide automatic 1C import or synchronise balances. Access is restricted to the Accountant.

FR12 Rental Price Estimation (Must)

The system shall provide the Property Owner and Rental Manager with an estimated total monthly rent in KZT based on area, location, property type, floor and renovation condition.

Acceptance criterion. Supported valid inputs return a non-negative monthly estimate through the application. Invalid or unsupported inputs produce a clear message. The result is labelled as an estimate and does not automatically change an agreement. Prediction service failure does not prevent core record management.

FR13 Model Training and Evaluation (Must)

The team shall prepare a rental dataset, train and compare Linear Regression and Random Forest models against a training-set median baseline, and integrate the selected model.

Acceptance criterion. The evaluation records the dataset source, target definition, features, split method, model settings and MAE and RMSE in KZT per month. Models and the median baseline use the same evaluation split. Dataset documentation identifies geography, collection period, supported types and whether the target is asking rent or concluded lease rent. A validation split or cross-validation supports model selection; final test data is not used for tuning. Preprocessing is fitted only on training data, and repeated listings for the same premise do not cross training, validation and test subsets. Model selection is explained using measured results.

6.5 Non-Functional Requirements

NFR01 Security (Must)

The system shall protect credentials and restrict protected data and operations by role.

Acceptance criterion. Passwords are stored as salted secure hashes rather than plain text. Deployment uses HTTPS. API tests reject unauthenticated and unauthorised requests. Credentials and secret keys are absent from the repository.

NFR02 Data Integrity (Must)

Related agreement and payment updates shall be processed consistently.

Acceptance criterion. A simulated failed operation leaves no partially saved changes. Concurrent lease requests cannot create overlapping confirmed agreements. Repeated payment submissions cannot create duplicate payment records. Payment corrections preserve traceability, and all property and agreement references remain valid.

NFR03 Usability (Must)

The interface shall provide clear forms, validation messages and navigation for core tasks.

Acceptance criterion. Each role can complete its assigned core scenario using the interface: the Property Owner reviews occupancy and rental revenue, the Rental Manager creates a tenant and agreement, and the Accountant records a payment and checks its balance. Required fields and invalid inputs are clearly identified.

NFR04 Browser Compatibility (Must)

The web interface shall support Chrome and Edge.

Acceptance criterion. Login, record management, agreement creation, payment recording and dashboard viewing complete successfully in both browsers. Test evidence records browser versions; controls and messages remain readable and usable.

NFR05 Reproducible Deployment (Must)

The project shall include instructions for installing, configuring and starting its components.

Acceptance criterion. Another team member follows the README in a clean environment to start the React frontend, Java/Spring Boot backend, relational database and ML service and load demonstration data. An end-to-end scenario saves a lease through the UI and REST API, reloads it from the database, registers a payment, updates dashboard totals and requests an ML estimate through the backend. The instructions identify required versions and settings. No bank or 1C connection is required to run the platform.

6.6 Basis and Verification

The Project Proposal defines the functional scope; the updated Literature Review supports the operational workflow and evaluation design. Literature Review Sections 3.1–3.2 support linked rental records and a manual accounting boundary (FR02–FR08 and FR10–FR11). Section 3.3 supports rental estimation, the median baseline and leakage-aware evaluation (FR12–FR13). Section 3.4 supports three-role access, transaction consistency and traceable payment entry (FR01, FR06–FR07 and NFR01–NFR02). Section 3.5 supports comparison with the current workflow. Rental revenue analytics in FR10 implement the original objective of operational financial reporting without introducing an operating expense or profitability module. These are documented design choices, not completed interviews or measured improvements.

Functional tests shall reference FR01–FR13, including conditional testing of Should export FR11, and quality checks shall reference NFR01–NFR05. Unit tests cover balances, date boundaries and validation. Integration tests cover database consistency, role permissions, overlapping lease requests, repeated payment operations, manual payment reconciliation and the prediction API. Interface tests cover the core scenarios for all three roles.

The test report shall record inputs, expected and actual results, pass or fail status, and the tested application version. ML results shall be reported separately using MAE and RMSE. Requirements and acceptance criteria describe planned checks and do not claim that implementation or testing has already been completed.

6.7 Alignment with the Project Proposal

The following mapping covers the expected outcomes in Proposal Section 1.7 and the corresponding objectives in Section 1.6. Outcome numbering refers to the Proposal; the profitability wording in Objective 8 and Outcome 7 must be updated to rental revenue analytics to match the agreed scope.

Outcome 1: Centralised Database. FR02–FR08 and FR10; NFR02 and NFR05 verify persistent linked records.

Outcomes 2 and 3: Properties, Premises and Tenants. FR02–FR04 cover creation, editing, search, filtering, contact details and related agreements.

Outcome 4: Lease Agreements. FR05–FR06 cover agreement management, availability checks and overlap prevention.

Outcome 5: Rental Payments. FR07–FR08 cover charges, manually verified payments, outstanding balances and arrears.

Outcome 6: Automated Notifications. FR09 covers scheduled expiry and overdue-payment monitoring.

Outcome 7: Analytical Dashboard. FR10 covers occupancy, area utilisation, rental charges, registered receipts, agreement status and outstanding balances. No operating expense or profitability module is required.

Outcome 8: Custom ML Module. FR12–FR13 cover data preparation, features, model comparison, MAE/RMSE and integrated estimates.

Outcome 9: Authentication and Role Access. FR01 and NFR01 cover the three-role model and security checks.

Outcomes 10 and 11: Integrated Application and Evidence. NFR05 and the verification plan cover the React, Java/Spring Boot, database and ML components, REST interactions, tests and documentation.

The scope exclusions in Proposal Section 1.8 are retained: no mobile application, online payment processing, full accounting, direct 1C integration or training of a large language model from scratch. Optional export FR11 is a Should extension, not a prerequisite for completing the core prototype.

7 Architecture and Diagrams

7.1 System Context Diagram

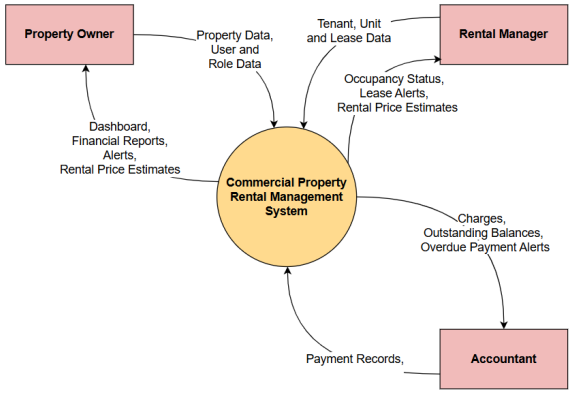


Figure 1: System Context Diagram

7.2 Use Case Diagram

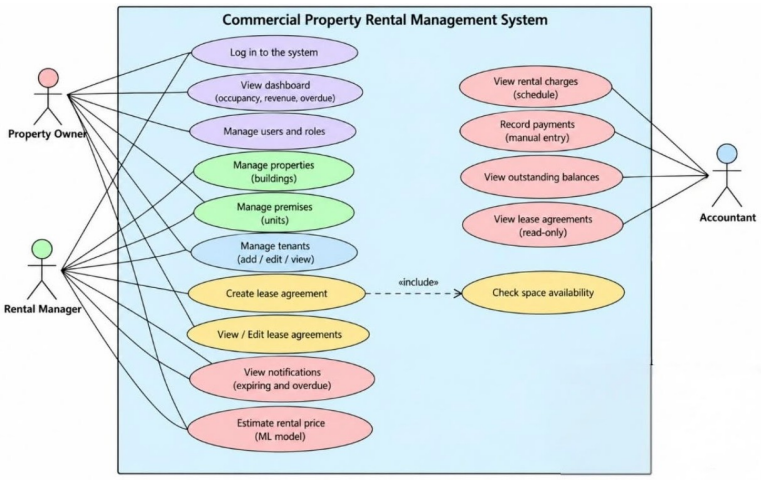


Figure 2: Use Case Diagram

7.3 Component Diagram

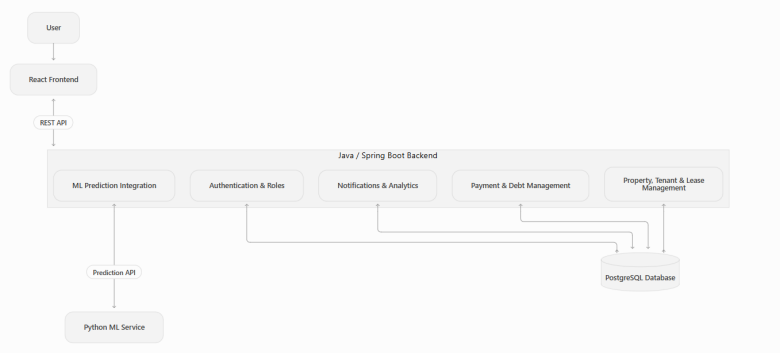


Figure 3: Component Diagram of the Commercial Property Rental Management System

8 Verification and Evaluation Plan

8.1 Purpose

Verification checks whether the application meets FR01-FR13 and NFR01-NFR05 of the requirements specification. It covers rental records, agreement availability, payment calculations, expense records, property profitability, scheduled notifications, dashboards, manual registration of confirmed payments and rental price prediction. Evaluation compares core work in the application with manual work in Excel and measures prediction errors separately.

The plan follows the updated Literature Review: connected operational records, accounting kept separately in 1C, external payment verification by the Accountant and manual entry into the platform. No direct 1C or bank integration is tested because none is in scope. FR01-FR10, FR12-FR13 and NFR01-NFR05 are Must; optional export FR11 is Should.

8.2 Functional verification

FR01 Authentication and role permissions

Test valid and invalid login, logout, blocked accounts and direct API access. Use Property Owner, Rental Manager and Accountant accounts. The Owner can review profitability; the Accountant can enter expenses. The Rental Manager cannot edit payments or expenses, and the Accountant cannot modify lease terms. Check permitted actions and reject prohibited actions, including account administration by the Rental Manager or Accountant. Unauthenticated requests must not return protected data.

FR02-FR04 Properties, premises, tenants and search

Create and update offices, retail spaces and warehouses. Verify property name, address, location and type. Reject duplicate premise codes within a property, invalid areas and incomplete tenant profiles. Verify links between tenants and agreements. Search properties by name and address and apply location and type filters. Search tenants, premises and agreements by name, code and agreement number; filters must return only matching records for the selected period.

FR05-FR06 Agreements and availability

Verify that unavailable periods are dated and excluded from searches and cannot conflict with confirmed leases. Test agreement creation and editing, required references, unique numbers, positive rent and valid dates. For a confirmed lease from 1 to 31 March inclusive, reject another lease starting 31 March and allow one starting 1 April if otherwise available. Verify that drafts do not reserve premises, confirmation reserves the lease period and expiry follows the end date. Repeat checks after edits. Submit simultaneous conflicting requests and confirm that no more than one succeeds.

FR07-FR08 Charges, payments and arrears

Use independently prepared expected values. Reject zero or negative charges. For a charge of 100,000 KZT and a payment of 60,000 KZT, verify an unpaid balance of 40,000 KZT. A charge due on 10 March becomes overdue on 11 March when unpaid. Test full and partial payments, advances and repeated submissions. Record the supporting external reference, allocation, entry author and time. Repeat a submission and reuse a supporting reference to check duplicate prevention and warnings. Correct a record and verify retained old and new values and author. Reject allocations exceeding the payment or unpaid charge. An externally confirmed but unentered receipt must not change the operational balance.

FR09 Notifications

Set and record the automatic check schedule, lease reminder period, control date and timezone. Wait for the scheduled process without manually triggering a check and confirm event generation. Test an agreement outside the period, on its boundary and within it. Verify overdue notifications after unpaid charge due dates. Notifications must identify the correct record, respect role permissions and remain unique when the same check runs again.

FR10 Occupancy, expenses and property profitability

Verify occupancy and area utilisation against reference areas: 100 rentable square metres and 40 occupied produce 40% and 60 vacant. Exclude unavailable area; with no rentable area, display N/A. As Accountant, create a paid

operating expense with property, payment date, positive amount, purpose and supporting reference; reject incomplete or invalid records. Check traceable corrections and deny expense edits by other roles. For one property and period, 1,000,000 KZT registered receipts and 300,000 KZT expenses must produce a 700,000 KZT operational result and a 70% margin. With zero receipts, margin is N/A; with expenses exceeding receipts, the result is negative. Exclude other properties and out-of-period items. Count each transaction once. Use actual payment date rather than entry timestamp; derive payment property through the agreement and premise. Exclude deposits, taxes, financing and capital expenditure from the operational result. Verify Owner and Accountant access, receipt-date grouping and labels distinguishing operational indicators from accounting profit. Unentered bank or 1C transactions do not affect totals.

FR11 Optional Excel or CSV export

If implemented, export payments for a defined period using the Accountant role. Compare identifiers, dates, agreements, amounts, currency, supporting references, record count and totals with the source records. Repeat export and check stable identifiers and unchanged source data. Reject access by other roles. If deferred, mark Should FR11 as not implemented; do not describe export as 1C import or synchronisation.

FR12 Rental price prediction interface

Use Property Owner and Rental Manager accounts to request predictions with supported premise characteristics. Verify a non-negative total monthly estimate in KZT and its advisory label. Invalid or unsupported inputs must produce a clear message. Confirm that prediction does not automatically change agreement rent and that core record management works when the prediction service is unavailable.

FR13 Model training and evaluation

Check dataset source, target definition, features, preparation and split documentation. Compare Linear Regression, Random Forest and a training-set median baseline on the same held-out data. Fit preprocessing on training data only and prevent repeated listings of the same premise from crossing training, validation and test subsets. Record random seed, model settings, sample counts, MAE and RMSE. Final test data must not guide model selection or tuning.

8.3 Quality verification

NFR01 Security

Inspect password storage for salted secure hashes, check HTTPS deployment and test unauthenticated and prohibited API requests. Inspect tracked project files for credentials and secret keys.

NFR02 Data integrity

Simulate failures during agreement, payment and expense updates. Verify that payment and expense corrections preserve their history. Verify rollback rather than partially saved records. Repeat conflicting lease requests and duplicate payment submissions to confirm consistency.

NFR03 Usability

Run the core scenario for each role: the Property Owner reviews occupancy and profitability, the Rental Manager creates a tenant and agreement, and the Accountant records a payment and checks its balance. Record completion, errors, requested assistance and unclear controls. Verify required-field labels and validation messages.

NFR04 Browser compatibility

Run login, record management, agreement creation, payment recording and dashboard scenarios in Chrome and Edge. Record versions and screen resolution. Controls and messages must remain readable and usable.

NFR05 Reproducible deployment

A team member follows the README in a clean environment to start the React frontend, Java/Spring Boot backend, relational database and ML service and load demonstration data. Run an end-to-end REST scenario: create a lease in the UI, reload persisted data, register a payment and expense, verify dashboard totals and request a prediction through the backend. Record dependency versions, configuration steps and any missing instructions. Confirm that core operation requires no 1C or bank connection. If export is implemented, verify it separately under FR11.

8.4 Baseline and evaluation tasks

Compare the application with manual record keeping in Excel using equivalent tenant, premise, agreement, charge and payment records. Excel filters and formulas are allowed. Both methods use the same reference date and logically equivalent datasets.

Use five task types from the updated review: check availability for a requested period; create a valid agreement; identify and reject an overlapping agreement; register a confirmed partial payment and calculate the remaining balance; and reconcile an external statement with platform records to identify an omitted or duplicate payment. Run each task in both methods with the same expected answer. In Excel, rejecting an overlap means

identifying the conflict and declining to record it. Use different but equivalent variants to reduce memorisation.

8.5 Outcome metrics

Task completion time

Measure from the moment the participant receives the task and may begin interacting with the method to the moment they submit their final answer. Compare median times for correct, unassisted completions of each task type. Report the number of observations behind every median.

Errors and task success

Count incorrect premises, invalid agreements accepted, valid agreements rejected, incorrect allocations or balances, and missed or falsely reported reconciliation discrepancies against the answer key. Report total errors and errors per attempt to account for different attempt counts. A successful attempt has a correct final answer without assistance. Success rate equals successful attempts divided by all attempts, multiplied by 100%. Assisted or unfinished attempts remain in the denominator and are reported separately; they are excluded from the successful-completion time median.

Time reduction (%) = (median Excel time - median application time) / median Excel time x 100. Calculate this separately for each task type. If either method has no successful attempts, report the time comparison as unavailable. Do not hide unsuccessful attempts behind faster successful-completion times.

A workflow benefit is demonstrated for a task when the application has a lower median completion time, no higher errors per attempt and no lower success rate than Excel. For reconciliation, compare missed discrepancies and false alarms per attempt against the seeded answer key. Report each task separately; do not claim an overall improvement when results are mixed.

ML prediction errors

For the same held-out records, report MAE as the mean absolute difference between actual and predicted monthly rent, and RMSE as the square root of the mean squared difference. Both are reported in KZT per month. Include the test sample size and a median-rent baseline fitted on training data only as an evaluation reference. This baseline is not an additional application feature.

Select the model using training and validation results, then report final held-out performance. Lower errors indicate better predictions on the tested data. Do not state an accuracy percentage or claim improved prediction unless results support it. Report whether targets are asking rents or concluded lease rents, and describe applicable geography and property types.

8.6 Measurement conditions

Recruit representatives of the three target roles and record participant count, role and relevant experience before trials. Recruitment numbers are fixed be-

fore execution rather than presented as completed participation. The Rental Manager performs availability and agreement tasks; the Accountant performs payment and reconciliation tasks; the Property Owner reviews occupancy and profitability under NFR03. Each participant performs only tasks within their role permissions. Compare the same participants' results across methods for each task type. Team-only trials are labelled preliminary internal evaluation.

Use the same computer, equivalent tasks and equal familiarisation time. Alternate which method is used first across participants. Restore the initial dataset between trials. Record assistance and interruptions. Define one common time limit for each task type before testing and apply it equally to both methods; attempts reaching it are recorded as unfinished.

Use synthetic or authorised anonymised bank or accounting evidence alongside platform records. Seed known omitted and duplicate entries for reconciliation; do not connect to live banking or 1C. Record which discrepancies each participant identifies and any false alarms. Use records covering vacant, occupied and unavailable premises, current and future agreements, full and partial payments, overdue charges and advances. Record the dataset version, reference date, timezone, application version, browser, hardware and notification settings. For profitability, use a separate reference set of property-linked receipts and expenses with known totals, including zero receipts, losses and out-of-period entries. For ML, record source, collection period, target units and the split used. These are planned measurements; no results are claimed in advance.

8.7 Test evidence and completion

Use unit tests for balances, occupancy, operational result, margin and validation, integration tests for database operations, manual payment and expense workflows, REST interactions and the prediction API, and interface scenarios for user workflows. After a defect is fixed, rerun the failed check and affected regression scenarios.

Each test record contains: test ID; FR or NFR reference; role and preconditions; input data and steps; expected result; actual result; pass, fail or not tested status; application version; and supporting evidence. Keep workflow evaluation results and ML evaluation results alongside their datasets and calculation methods.

Verification is complete when the acceptance criteria for all mandatory requirements are met and defects causing overlapping leases, incorrect financial calculations, duplicate payments, data loss or access violations are resolved. Failed or untested mandatory checks remain visible and prevent a claim of full verification.

The final report presents requirement coverage, end-to-end integration evidence, financial calculation tests, workflow comparison, ML errors, identified defects and remaining limitations. Use the outcome mapping in specification section

7 to confirm coverage of all eleven expected outcomes in Proposal section 1.7. The expense and profitability checks implement Proposal outcome 7; the operational workflow and five baseline tasks follow the updated Literature Review. Optional FR11 is reported separately and does not block mandatory verification if deferred. No acceptance result implies automatic reconciliation with bank records or 1C. Evaluation conclusions must stay within the tested users, data and conditions.

9 Project Management and Risks

9.1 Risk Management

Risk management is an important part of the project planning process. The team has identified three key risks that may affect the development of the Commercial Property Rental Management System.

Risk Register

ID	Risk Description	Likelihood	Impact	Risk Owner
R1	Insufficient or low-quality ML training data	High	High	Aluana Muslimova
R2	Data exchange limitations with 1C	Medium	Medium	Ayaulym Idrissova
R3	Errors in outstanding balance calculations and lease agreement validation	Medium	High	Aruzhan Seitkhankyzy

Table 8: Project Risk Register

Risk Mitigation Strategies

R1. Insufficient ML Training Data The team will investigate multiple data sources, clean and standardize the dataset, and compare Linear Regression and Random Forest models.

If sufficient data cannot be obtained, the prediction scope will be limited to property categories with more reliable data.

R2. Data Exchange Limitations with 1C The team will consider exporting rental management data in Excel or CSV formats.

Direct integration with 1C is outside the project scope. If compatible data exchange is not possible, verified payments will be recorded manually.

R3. Errors in Business Logic The team will define clear business rules and conduct unit and integration testing.

Special attention will be given to partial payments, overdue balances, lease expiration dates, and overlapping rental periods.

Risk Monitoring and Control

The identified risks will be monitored throughout the development process using GitHub and YouTrack.

Each risk will have an assigned responsible team member. Risk assessments will be reviewed at major project milestones, and mitigation measures will be updated when necessary.

9.2 Project Milestones

Milestone	**Description**	**Expected Deliverables**
M1. Requirements Analysis	Analyze user needs, identify system requirements, and study existing solutions.	Requirements specification and project proposal.
M2. System Design	Design the system architecture, database structure, and user interface.	Architecture diagrams, database design, and UI/UX prototype.
M3. Data Collection and Preparation	Collect, clean, and prepare commercial property rental data for machine learning.	Prepared dataset for ML model training.
M4. Backend Development	Develop the database, REST APIs, authentication, and rental management business logic.	Functional backend with core system operations.
M5. Frontend Development	Develop the web application interface using React, JavaScript, HTML, CSS, and Bootstrap.	Functional user interface with the main rental management modules.
M6. ML Model Development	Train and evaluate machine learning models for rental price prediction.	Trained ML model and evaluation results.
M7. System Integration and Testing	Integrate all system components and test their functionality.	Integrated and tested web application.
M8. Final Documentation	Prepare technical documentation and the final project presentation.	Final documentation, working prototype, and presentation.

Table 9: Project Milestones

9.3 Project Dependencies

The project consists of several connected stages. Some tasks depend on the completion of others, while some activities can be carried out at the same time.

Project Stage	**Depends On**	**Description**
Requirements Analysis	–	The first stage where the team identifies system requirements.
System Design	Requirements Analysis	The system architecture and database are designed based on the requirements.
Data Collection	Requirements Analysis	Data for the ML model is collected based on the required property characteristics.
Backend Development	System Design	The backend is developed based on the system architecture and database design.
Frontend Development	System Design	The interface is developed based on the UI/UX design and planned API structure.
ML Model Development	Data Collection	The ML model is trained using the prepared dataset.
Integration and Testing	Backend, Frontend, ML Model	All components are connected and tested together.
Final Documentation	Integration and Testing	The final documentation is completed after testing the system.

Table 10: Project Dependencies

Frontend and backend development can be done in parallel after the system design stage. Data collection for the ML model can also begin during requirements analysis to avoid delays.

These dependencies help the team organize tasks and complete the project in the correct order.

References

[1] Bassett, R., & Pisano, M. (2022). Property Management – An Industry of Fragmented Technology Solutions. *Information Technology and Management Science*, 25, 8–15. DOI: 10.7250/itms-2022-0002.

Read the full text. See pp. 10–11, Leasing Process and Rent Collection and Financials sections.

[2] Pomè, A. P., Tagliaro, C., & Lavy, S. (2026). The PropTech Mismatch: Exploring the Demand–Supply Digital Gap in Italy's Property Management Sector. *Sustainability*, 18(4), 1850. DOI: 10.3390/su18041850.

Read the full text. See sections 3.2–3.4, 5 and 6. Alternative university repository page.

[3] Wang, F. (2023). The present and future of the digital transformation of real estate: A systematic review of smart real estate. *Business Informatics*, 17(2), 85–97. DOI: 10.17323/2587-814X.2023.2.85.97.

Read the full text. See the sample description, bibliometric results and conclusion.

[4] Mohd, T., Harussani, M., & Masrom, S. (2022). Rapid Modelling of Machine Learning in Predicting Office Rental Price. *International Journal of Advanced Computer Science and Applications*, 13(12), 543–549. DOI: 10.14569/IJACSA.2022.0131266.

Read the full text. See section III and Table VI on p. 548.

[5] Mohd, T., Harussani, M., Masrom, S., Ab Rahman, H., & Rahman, L. (2023). Office Building Rental Prediction Model Based on Locational Determinants. *International Journal of Sustainable Construction Engineering and Technology*, 14(3), 229–238.

Read the full text. See sections 3.3, 4 and 5.

[6] Deppner, J., & Cajias, M. (2024). Accounting for Spatial Autocorrelation in Algorithm-Driven Hedonic Models: A Spatial Cross-Validation Approach. *The Journal of Real Estate Finance and Economics*, 68, 235–273. DOI: 10.1007/s11146-022-09915-y. First published online in 2022.

Read the full text. See Data and Methodology, Results and Conclusion. The page includes a Download PDF option.

[7] Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-Based Access Control Models. *Computer*, 29(2), 38–47.

Read the full text. Open author version hosted by NIST; see sections 1–4.

[8] Berenson, H., Bernstein, P., Gray, J., Melton, J., O'Neil, E., & O'Neil, P. (1995). A Critique of ANSI SQL Isolation Levels. *Proceedings of ACM SIGMOD*, 1–10.

Read the full text. Microsoft Research repository, MSR-TR-95-51; see sections 2–5.

[9] scikit-learn developers. *DummyRegressor*. Official documentation. Accessed 18 October 2026.

Read the documentation. See the `strategy` parameter and the `median` option.

[10] scikit-learn developers. *Common pitfalls and recommended practices*. Official documentation. Accessed 18 October 2026.

Read the documentation. See Inconsistent preprocessing and Data leakage.

[11] scikit-learn developers. *Metrics and scoring: quantifying the quality of predictions*. Official documentation. Accessed 18 October 2026.

Read the documentation. See Regression metrics, including MAE and RMSE.

[12] PostgreSQL Global Development Group. *Range Types*. Official PostgreSQL documentation. Accessed 18 October 2026.

Read the documentation. See Constraints on Ranges and exclusion constraint examples.

[13] 1C:First Bit, Almaty. (2021). *Automation of management accounting at Dars-2 using 1C:Accounting for Kazakhstan*. 1C Solutions. Implementation case, February 2021. Title translated from Russian. Accessed 21 October 2026.

Read the full case. See Project description, Key project results and the director's statement. Public vendor report, not a scientific publication.

[14] Astana Hub. *Development of CRM/WMS (Razrabotka CRM/WMS)*. Business request for the development of an information system for commercial warehouse and industrial property management. Original title in Russian. Accessed 21 October 2026.

View the original business request. Public business request, not a scientific publication.

[15] React. *Managing State*. [Online]. Available: https://react.dev/learn/managing-state. Accessed: Oct. 21, 2026.

[16] Vue.js. *State Management*. [Online]. Available: https://vuejs.org/guide/scaling-up/state-management.html. Accessed: Oct. 21, 2026.

[17] Angular. *Reactive Forms*. [Online]. Available: https://angular.dev/guide/forms/reactive-forms. Accessed: Oct. 21, 2026.

[18] Spring. *Managing Transactions*. [Online]. Available: https://spring.io/guides/gs/managing-transactions/. Accessed: Oct. 21, 2026.

[19] Spring Security. *Method Security*. [Online]. Available: https://docs.spring.io/spring-security/reference/servlet/authorization/method-security.html. Accessed: Oct. 21, 2026.

[20] Django Software Foundation. *Database Transactions*. Django 5.2 documentation. [Online]. Available: https://docs.djangoproject.com/en/5.2/topics/db/transactions/. Accessed: Oct. 21, 2026.

[21] Django Software Foundation. *Using the Django Authentication System*. Django 5.2 documentation. [Online]. Available: https://docs.djangoproject.com/en/5.2/topics/auth/default/. Accessed: Oct. 21, 2026.

[22] Express. *Database Integration*. [Online]. Available: https://expressjs.com/en/guide/database-integration/. Accessed: Oct. 21, 2026.

[23] PostgreSQL Global Development Group. *Range Types*. PostgreSQL documentation, Sec. 8.17. [Online]. Available: https://www.postgresql.org/docs/current/rangetypes.html. Accessed: Oct. 21, 2026.

[24] Oracle. *Locking Reads*. MySQL 8.0 Reference Manual, Sec. 17.7.2.4. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/innodb-locking-reads.html. Accessed: Oct. 21, 2026.

[25] MongoDB. *Atomicity and Transactions*. MongoDB documentation. [Online]. Available: https://www.mongodb.com/docs/manual/core/write-operations-atomicity/. Accessed: Oct. 21, 2026.